

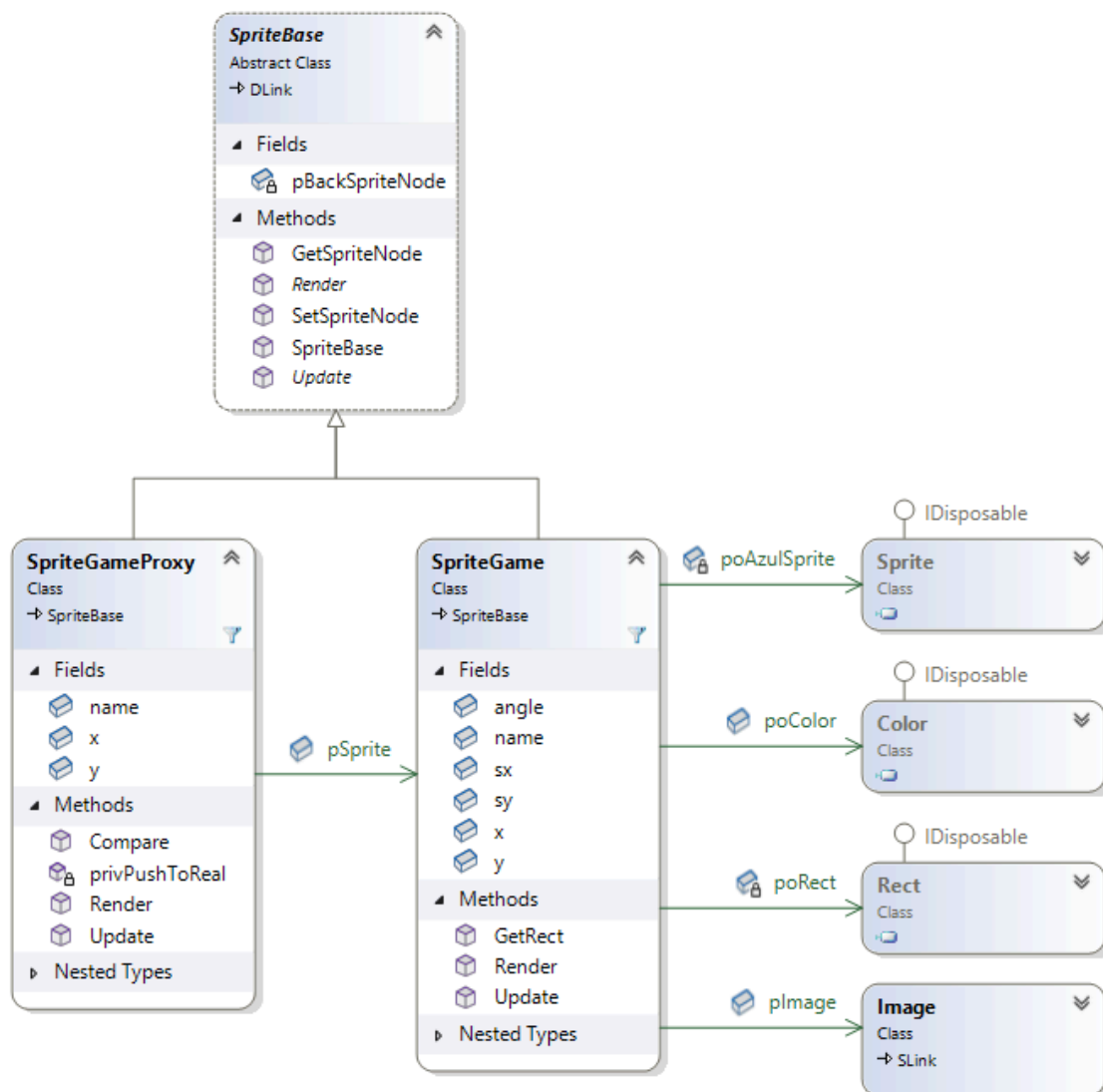
Proxy Pattern

Problem:

The alien grid in Space Invaders consists of 3 alien types: squids, octopi, and crabs. The total number of aliens in the grid is 55; creating game sprites for each alien is inefficient regarding memory usage and performance. We need to create a system to reuse the same 3 sprites to fill up the 55 aliens in the grid while also maintaining differences in their positions on the screen.

Solution:

We can go about reducing the number of heavy data objects, like game sprites, via the use of proxies that are more simple and lightweight. The proxy pattern allows us to create multiple sprite proxies that can all point to a real game sprite. These proxies can store unique x and y positions that we can then push onto the real game sprite. We will use them as a mediator that can deal with all the functionalities by passing them to the real game sprite.



Pattern Description:

By acting as a bridge between a client and an actual object, the proxy pattern gives access to the real object more control and a degree of indirection. This can serve multiple purposes, like restricting access privileges, postponing the creation or loading of the actual object until required, or offering extra functionality in relation to the functions of the actual object.

The proxy objects push their data, like coordinates, to the matching real game sprite objects when it's time to render the game sprites. This keeps the system design light and effective while guaranteeing that the sprites are shown on the screen as intended.

Key Object-Oriented Mechanics:

In terms of object-oriented mechanics, the proxy pattern leverages key principles such as abstraction and encapsulation. Abstraction is employed here through the definition of an interface shared by both the proxy and the real object, ensuring that we can interact with them using a unified set of methods (this is done through the use of a common parent class, Sprite Base). Encapsulation plays a crucial role in concealing the complexities of the real object behind the proxy, enabling the proxy to manage interactions with the real object internally.

Usage in Space Invaders:

Lightweight proxy objects are used to interact with game sprite objects instead of the heavy-weight ones directly. These proxies store important information, such as the location of specific aliens, and when needed, they transfer requests to the actual game sprite objects. This makes it possible to handle game sprite operations in a more optimized and regulated manner.

Each proxy holds the location of one alien (x, y). To display these aliens in the correct location on our screen, we use the proxies to push the coordinates onto the real game sprite and then render them. We only store and push important data to keep the proxies light in the data. Since multiple proxies can point to the same real game sprite, we only keep one of each alien (squid, octopus, and crab) in the game sprites. A similar implementation is done to render multiple sets of shield bricks that the ship can use for cover. Something to note here is that the sprites need to have the coordinates of the proxy pushed to them before it is rendered to get the correct placement of the game sprites.